

ODDELEK ZA FIZIKO, FMF, UL, 2023/2024

MATEMATIČNO-FIZIKALNI PRAKTIKUM, 3. letnik

Naloga 12: Strojno Učenje

AVTOR:
Dimitrije Pešić

MENTOR:
prof. dr. Borut Paul Kerševan

Ljubljana, Marec 2024

Vsebina

1	Uvod	1
2	Ozadje	1
3	MNIST	4
4	Predobdelava podatkov	4
4.1	Ocena modelov	6
5	TensorFlow implementacija	7
5.1	Pretreniranje in podtreniranje	7
5.2	Primerjava aktivacij	8
5.3	Primerjava optimizatorjev	10
5.4	Osnovne in izpeljane spremenljivke	11
6	Pospešena odločitvena drevesa (BDT)	12
6.1	Primerjava velikosti	12
6.2	Osnovne in izpeljane spremenljivke	13
6.3	Optimalno drevo	14
7	PyTorch implementacija	16
7.1	Optimalen	18
8	Zaključek	19

1 Uvod

Na spletni učilnici smo dobili na voljo material (koda, vzorci) za ločevanje dogodkov Higgsovega bozona od ostalih procesov ozadja. V naboru simuliranih dogodkov je 18 karakteristik (zveznih kinematičnih lastnosti), katerih vsaka posamezno zelo slabo loči 'signal' od ozadja, z uporabo BDT ali (D)NN, pa lahko tu dosežemo zelo dober uspeh. Na predavanjih smo si ogledali glavne aspekte pomembne pri implementaciji ML, kot so uporaba ustreznih spremenljivk (GIGO), učenje in prekomerno učenje (training/overtraining), vrednotenje uspeha metode kot razmerje med učinkovitostjo (efficiency) in čistostjo (precision) vzorca (Receiver Operating Characteristic, ROC).

Naloga je od nas zahtevala, da določimo uspešnost obeh metod (in narišemo ROC) za nekaj tipičnih konfiguracij BDT in DNN, pri čemer naj:

- Študiramo vpliv uporabljenih vhodnih spremenljivk - kaj, če vzamemo le nekatere?
- Študiramo BDT in NN in vrednoti uspešnost različnih nastavitev, če spreminjaš nekaj konfiguracijskih parametrov (npr. število perceptronov in plasti nevronske mreže pri DNN in število dreves pri BDT).

2 Ozadje

Dandanes je uporaba različnih algoritmov strojnega učenja (Machine Learning, ML) v znanosti že rutinsko opravilo. Poznamo tri osnovne vrste stojnega učenja:

- Nadzorovano učenje (Supervised learning):
 - Klasifikacija (Classification): sortiranje v različne kategorije.
 - Regresija (Regression): modeliranje oz. 'fitanje' napovedi.
- Nenadzorovano učenje (npr. sam najdi kategorije).
- Stimulirano učenje (Artificial Intelligence v ožjem pomenu besede).

V fiziki (in tej nalogi), se tipično ukvarjamo s prvo kategorijo, bodisi za identifikacijo novih pojavov/delcev/... ali pa za ekstrakcijo napovedi (netrivialnih funkcijskih odvisnosti etc).

ML algoritmi imajo prednost pred klasičnim pristopom, da lahko učinkovito razdrobijo kompleksen problem na enostavne elemente in ga ustrezno opišejo:

- pomisli na primer, kako bi bilo težko kar predpostaviti/uganiti pravo analitično funkcijo v več dimenzijah (in je npr. uporaba zlepkov (spline interpolacija) mnogo lažja in boljša).

- Pri izbiri/filtriranju velike količine podatkov z mnogo lastnostmi (npr dogodki pri trkih na LHC) je zelo težko najti količine, ki optimalno ločijo signal od ozadja, upoštevati vse korelacije in najti optimalno kombinacijo le-teh. . .

Če dodamo malce matematičnega formalizma strojnega učenja: Predpostavi, da imamo na voljo nabor primerov $\mathcal{D} = \{(\mathbf{x}_k, y_k)\}_{k=1..N}$, kjer je $\mathbf{x}_k = (x_k^1, \dots, x_k^M)$ naključno izbrani vektor M lastnosti (karakteristik) in je $\mathbf{y}_k = (y_k^1, \dots, y_k^Q)$ vektor Q ciljnih vrednosti, ki so lahko bodisi binarne ali pa realna števila¹. Vrednosti $(\mathbf{x}_k, \mathbf{y}_k)$ so neodvisne in porazdeljene po neki neznani porazdelitvi $P(\cdot, \cdot)$. Cilj ML metode je določiti (priučiti) funkcijo $h : \mathbb{R}^Q \rightarrow \mathbb{R}$, ki minimizira pričakovano vrednost *funkcije izgube* (*expected loss*)

$$\mathcal{L}(h) = \mathbb{E} L(\mathbf{y}, \mathbf{h}(\mathbf{x})) = \frac{1}{N} \sum_{k=1}^N L(\mathbf{y}_k, \mathbf{h}(\mathbf{x}_k)).$$

Tu je $L(\cdot, \cdot)$ gladka funkcija, ki opisuje oceno za kvaliteto napovedi, pri čemer so vrednosti $(\mathbf{x}, \mathbf{y})$ neodvisno vzorčene iz nabora $\mathcal{D}$ po porazdelitvi P . Po koncu učenja imamo torej na voljo funkcijo $\mathbf{h}(\mathbf{x})$, ki nam za nek vhodni nabor vrednosti $\hat{\mathbf{x}}$ poda napoved $\hat{\mathbf{y}} = \mathbf{h}(\hat{\mathbf{x}})$, ki ustrezno kategorizira ta nabor vrednosti.

Funkcije $\mathbf{h}$ so v praksi sestavljene iz (množice) preprostih funkcij z (nekaj) prostimi parametri, kar na koncu seveda pomeni velik skupni nabor neznanih parametrov in zahteven postopek minimizacije funkcije izgube.

Osnovni gradnik odločitvenih dreves je tako kar stopničasta funkcija $H(x_i - t_i) = 0, 1$, ki je enaka ena za $x_i > t_i$ in nič drugače in kjer je x_i ena izmed karakteristik in t_i neznani parameter. Iz skupine takšnih funkcij, ki predstavljajo binarne odločitve lahko skonstruiramo končno uteženo funkcijo

$$\mathbf{h}(\mathbf{x}) = \sum_{i=1}^J \mathbf{a}_i H(x_i - t_i),$$

kjer so $\mathbf{a}_i$ vektorji neznanih uteži. Tako t_i kot $\mathbf{b}_i$, lahko določimo v procesu učenja. Nadgradnjo predstavljajo nato *pospešena* odločitvena drevesa (BDT), kjer nadomestimo napoved enega drevesa z uteženo množico le-teh, tipično dobljeno v ustreznih iterativnih postopkih (npr. AdaBoost, Gradient Boost ipd.).

Pri nevronskih mrežah je osnovni gradnik t.i. *perceptron*, ki ga opisuje preprosta funkcija

$$h_{w,b}(\mathbf{X}) = \theta(\mathbf{w}^T \cdot \mathbf{X} + b),$$

kjer je $\mathbf{X}$ nabor vhodnih vrednosti, $\mathbf{w}$ vektor vrednosti uteži, s katerimi tvorimo uteženo vsoto ter b dodatni konstatni premik (bias). Funkcija θ je preprosta gladka funkcija (npr. arctan), ki lahko vpelje nelinearnost v odzivu perceptrona. Nevronska mreža je nato sestavljena iz (poljubne) topologije takšnih perceptronov, ki na začetku sprejme karakteristiko dogodka $\mathbf{x}$ v končni fazi rezultirajo

¹...ali pa še kaj, prevedljive na te možnosti, npr barve. . .

v napovedi $\hat{\mathbf{y}}$, ki mora seveda biti čim bližje ciljni vrednosti $\mathbf{y}$. Z uporabo ustrezne funkcije izgube (npr MSE: $\mathcal{L}(h) = \mathbb{E} \|\mathbf{y} - \hat{\mathbf{y}}\|^2$), se problem znova prevede na minimizacijo, kjer iščemo optimalne vrednosti (velikega) nabora uteži $\mathbf{w}_i$ ter b_i za vse perceptrone v mreži. Globoke nevronske mreže (DNN) niso nič drugega, kot velike nevronske mreže ali skupine le-teh.

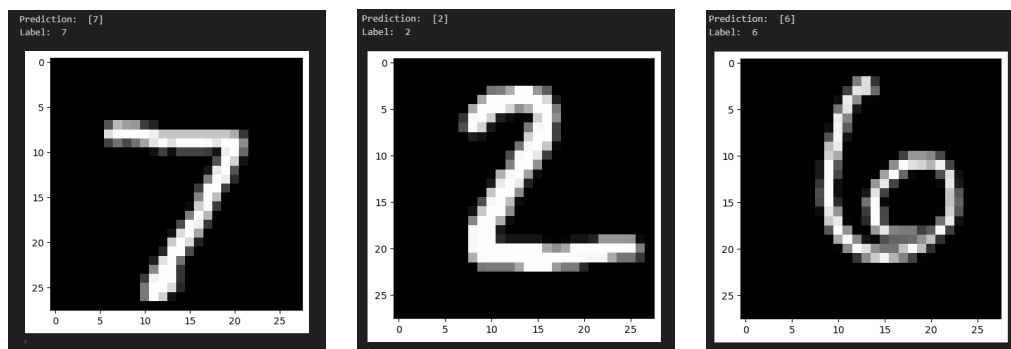
Že namizni računalniki so dovolj močni za osnovne računske naloge, obstajajo pa tudi že zelo uporabniku prijazni vmesniki v jeziku Python, na primer:

- Scikit-Learn (*scikit-learn.org*): odprtokodni paket za strojno učenje,
- TensorFlow (*tensorflow.org*): odprtokodni Google-ov sistem za ML, s poudarkom na globokih nevronskih mrežah (Deep Neural Networks, DNN) z uporabo vmesnika Keras. Prilagojen za delo na GPU in TPU.
- Catboost: (*Catboost.ai*) : odprtokodna knjižnica za uporabo pospešenih odločitvenih dreves (Boosted Decision Trees, BDT). Prilagojena za delo na GPU.

Za potrebe naloge lahko uporabimo tudi spletni vmesnik Google Collab (*colab.research.google.com*), ki dopušča omejen dostop do večjih računskih zmogljivosti.

3 MNIST

Projekt sem začel z globokim potopom v zapletenost nevronske mreže, pri čemer se sprva nisem zavedal njihove popolne kompleksnosti. Začel sem z implementacijo preproste nevronske mreže v Pythonu, ki uporablja samo Numpy. Isto sem implementiral tudi v TensorFlow, Pytorch in Scikit-learn ogrodjih. Poskusil sem tudi implementacijo v C++ programskem jeziku, vendar mi je vmes zmanjkalo časa ter znanja. Te modele sem nato apliciral na priljubljen nabor podatkov za treniranje modelov strojnega učenja - MNIST, to je nabor ki je sestavljen iz označenih črno belih slik ročno napisanih števk. Spodaj je prikazanih nekaj pravilno klasificiranih slik z uporabo po meri izdelane nevronske mreže [1](#). Raziskave so pokazale, da klasifikacijski modeli niso tako uspešni pri slikah kot so na primer konvolucijske mreže, vendar to presega obseg projekta.



Slika 1: Prikaz klasifikacije na naboru podatkov MNIST.

4 Predobdelava podatkov

Predobdelava (angl.: *preprocessing*) podatkov je morda kar najpomembnejši korak pri treniranju modelov strojnega učenja, saj kakovost vhodnih podatkov močno vpliva na uspešnost modela. Pravilna priprava in transformacija podatkov zagotavlja, da se model lahko učinkovito uči in generalizira iz podanih značilnosti, kar zmanjša šum in izboljša natančnost napovedi. Ključna je velika količina kvalitetnih označenih podatkov ter velika računalniška zmogljivost, da lahko model treniramo.

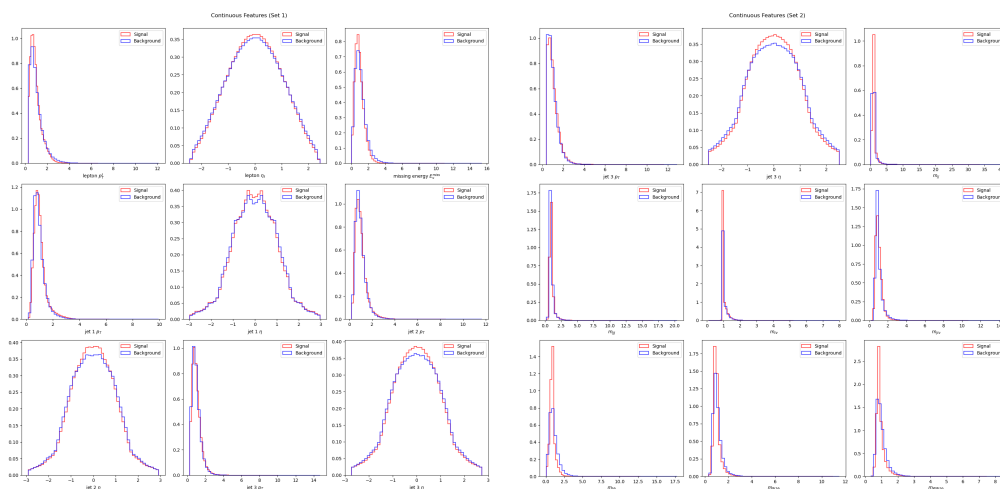
Pri tej nalogi smo se morali soočiti s podatkovnim setom odkritja Higgsovega bozona. Ta set podatkov, vključuje simulirane podatke trkov protonov pri invarijantni masi $\sqrt{s} = 8$ TeV. Vsebuje 28 spremenljivk, ki zajemajo različne merljive vrednosti interakcij delcev, opaženih pri teh trkih. Te spremenljivke vključujejo meritve leptonov, kot sta transversalna gibalna količina (p_T) in pseudorapidnost (η)², pljuske (z informacijami o prečnoprečni gibalni količini,

²Pseudorapidnost (η) je definirana kot $\eta = -\ln(\tan(\theta/2))$.

pseudorapidnosti in b-tagging³⁾ ter manjkajočo energijo.

Poleg tega podatkovni set navaja invarijantne mase za različne kombinacije delcev, kot so pljuski, leptoni in nevtrini. Invarijantne mase so višje nivojske spremenljivke, pridobljene iz osnovnih spremenljivk, in odražajo maso sistemov delcev, ki ostajajo nespremenjeni pri Lorentzovih transformacijah. Te pridobljene spremenljivke so ključne za razumevanje kompleksnih interakcij in so pomembne za razlikovanje med signali Higgsovih bozonov in ozadjem. Na slikah 2 se nahajajo grafi vseh zveznih spremenljivk neobdelanih podatkov, na grafu 3 pa preostanek.

Čeprav je običajno priporočljivo osredotočiti se na zvezne spremenljivke za treniranje modelov strojnega učenja, sem se odločil za usposabljanje z vsemi razpoložljivimi podatki, vključno z diskretnimi in enotnimi spremenljivkami čeprav je to v nasprotju s načelom "Garbage In, Garbage Out" (GIGO), ki poudarja, da kakovost in celovitost vhodnih podatkov močno vplivata na uspešnost modela. Podatke sem dobil na prostodostopnem repositoriju [1], za podrobnejšo razlago Atlas eksperimenta in podatkov prikazanih tukaj predlagam prebiranje originalnega članka odkritja Higgsovega bozona [2] ter seminar [3].

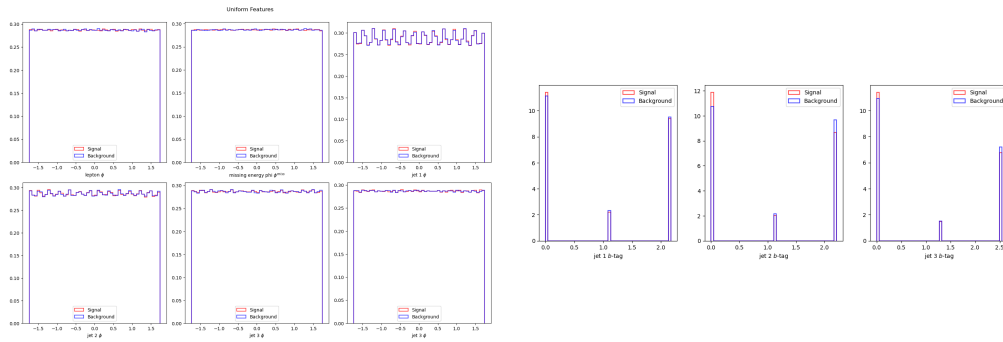


Slika 2: Histogrami porazdelitev zveznih neobdelanih podatkov. V rdečem so prikazani podatki za signal, v modrem pa za ozadje.

Preden sem podatke lahko uporabil je bil nabor podatkov predhodno obdelan prvo z normalizacijo, oznake pa so bile shranjene kot cela števila (0 za ozadje, 1 za signal). Čeprav ima celoten nabor podatkov 11 milijonov primerov, sem uporabil le majhen nabor 10.000 naključno izbranih vzorcev (da prihranim mučenje računalniku ter zaradi optimizacije časa - prepozno oddajanje naloge), razdeljenih na sklope za treniranje (80%) in validacijo (20%).

³B-tagging je metoda za zaznavanje prisotnosti b-kvarkov v pljusku.

4.1. Ocena modelov



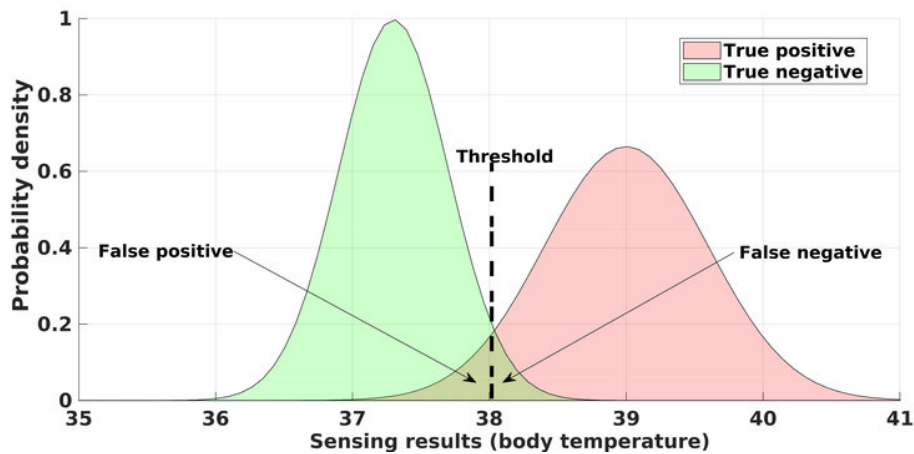
Slika 3: Histogrami porazdelitev uniformnih (levo) in diskretnih (desno) neobdelanih podatkov.

4.1 Ocena modelov

Vredno je spregovoriti besedo o temu kako oceniti uspešnost modelov. Za oceno uspešnosti modela smo uporabili Receiver Operating Characteristic (ROC) krivuljo, ki prikazuje razmerje med Deležem Pravilno Pozitivnih Primerov (TPR) in Deležem Lažno Pozitivnih Primerov (FPR) za različne vrednosti praga. TPR in FPR sta definirana kot:

- Delež pravilno pozitivnih primerov (TPR): $\frac{TP}{TP+FN}$
- Delež lažno pozitivnih primerov (FPR): $\frac{FP}{FP+TN}$

Kjer TP = pravilno pozitivni primeri, FP = lažno pozitivni primeri, TN = pravilno negativni primeri in FN = lažno negativni primeri. Vizualno željen rezultat prikazan na grafu 4



Slika 4: Skica ocenitvenega histograma.

5 TensorFlow implementacija

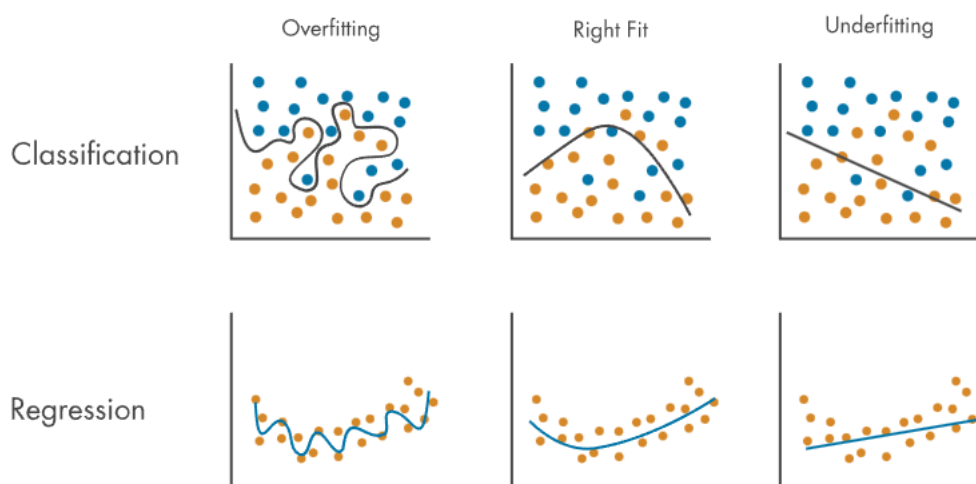
Kot je naloga zahtevala sem se prvo lotil implementacije (D)NN z uporabo TensorFlow-ovega Keras API za gradnjo modelov, kar pa sem kasneje obžaloval. Namreč izkaže se, da Tensorflow knjižica že nekaj časa ne podpira uporabo GPU na sistemih Windows, na čemu sem porabil kar precej časa poskušajoč usposobiti (tudi z uporabo WSL ne deluje). Posledično se tudi nisem lotil analize porabe procesorske moči ter porabljenega časa za treniranje, saj je razlika med uporabo CPU in GPU znatna. Nazoren prikaz razlike iz Nvidia konference [4].

5.1 Pretreniranje in podtreniranje

Ena izmed najtežjih stvari pri usposabljanju modelov umetne inteligence/strojnega učenja, je problem podtreniranja in pretreniranja:

- **Overfitting (prenaučenost)** nastopi, ko se model preveč dobro nauči učne podatke, vendar se ne more dobro posplošiti na nevidene podatke. To se običajno zgodi, ko model treniramo preveč epoh ali ko ima model preveč parametrov.
- **Underfitting (prenizko naučenost)** nastopi, ko je model preveč preprost, preveč regulariziran ali ni bil treniran dovolj dolgo in se ne more naučiti iz učnih podatkov. Cilj je najti pravo ravnotežje med tem, da treniramo dovolj dolgo, da se naučimo vzorce iz podatkov, in da se izognemo overfittingu z uporabo tehnik regularizacije.

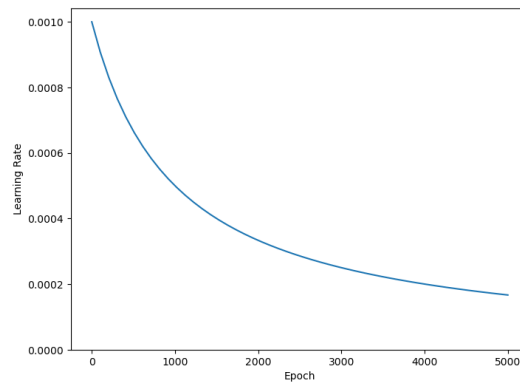
Prikaz problema na sliki 5.



Slika 5: Skica problema prenaučnosti in prenizke naučenosti.

Začnemo z majhnim modelom in eksperimentiramo z povečevanjem števila plasti in nevronov. Dodamo:

- **Načrtovalnik učne hitrosti:** dinamično prilagaja hitrost učenja med treningom, da izboljša uspešnost modela in konvergenco (prikazan na sliki 6).
- **Weight regularization (L2):** Regularizacija kaznuje velike uteži, kar pomaga omejiti kompleksnost modela.
- **Dropout:** Naključno opuščanje nevronov med učenjem prisili mrežo, da se nauči bolj generaliziranih vzorcev.
- **Kombiniranje L2 regularizacije in Dropout:** Ta kombinacija zagotavlja najboljše rezultate z uravnoteženjem kapacitete modela in regularizacije.



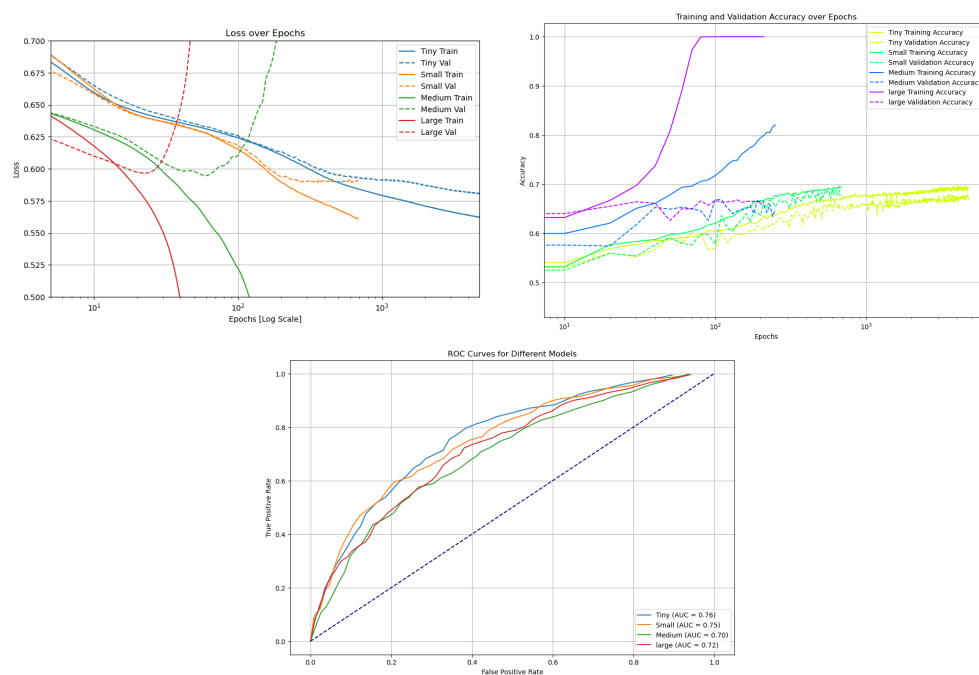
Slika 6: Skica načrtovalnika učne hitrosti.

Rezultati so prikazani na slikah 7. Opazimo da je najboljši model nekje vmes, med ta najmanjšim in največjim zato tega uporabljamo v nadaljevanju. Rezultati na ROC krivuljah kažejo na drugačno mnenje, vendar če smo dobro pozorni je razvidno, da najmanjši model ni najboljši, samo prepozno smo zaustavili učenje za večje modele.

5.2 Primerjava aktivacij

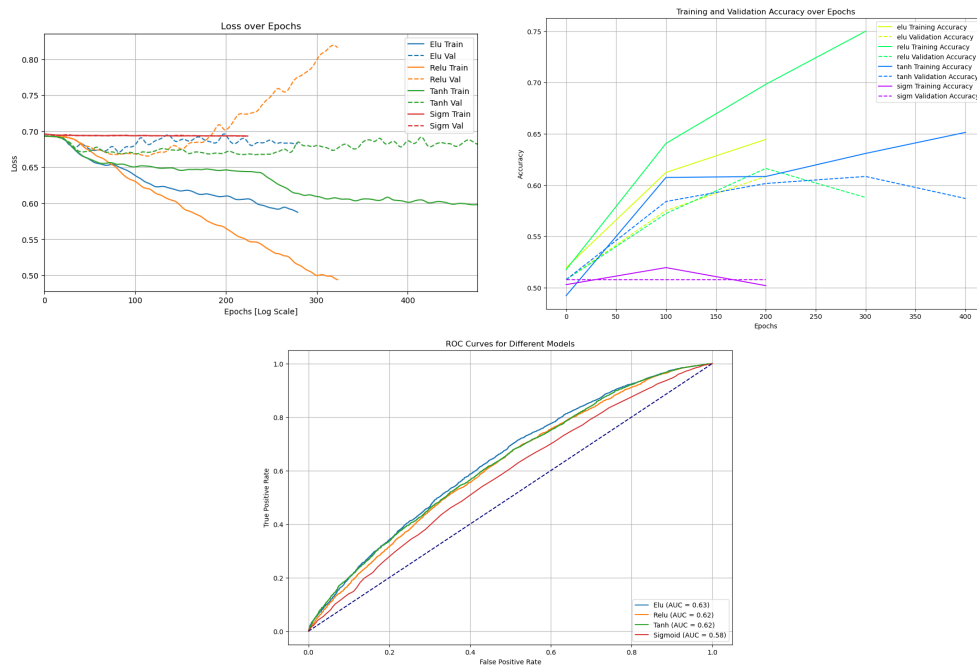
Preveril sem več aktivacijskih funkcij in optimizatorjev **??**. Testirane aktivacijske funkcije so bile ReLU, ELU, Tanh in Sigmoid. ReLU in ELU sta običajno nudili hitrejšo konvergenco in boljšo uspešnost v primerjavi s Tanh in Sigmoid, ki lahko trpita zaradi počasnejšega učenja in saturacije gradientov saj sta bolj kompleksne oblike.

5.2. Primerjava aktivacij

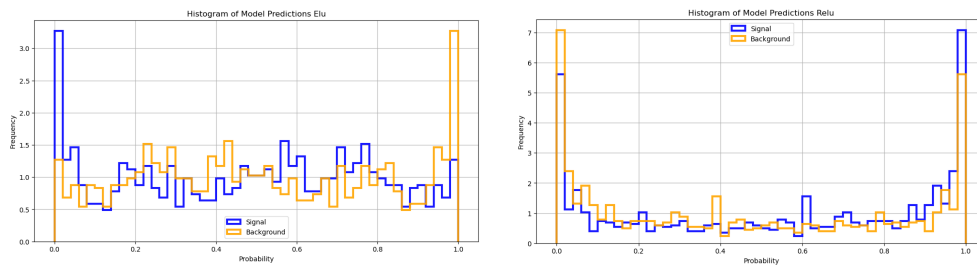


Slika 7: Prikaz izgube (levo), točnosti (sredina) in ROC + AUC (desno).

Tukaj mi je hitro postalo jasno, da nekaj ni povsem vredno ko sem preveril histogram uspešnosti modelov 9 in 10. Zelo očitno je da je klasifikacija popolnoma naključna ali pa še huje, popolnoma napačna.



Slika 8: Prikaz izgube (levo), točnosti (sredina) in ROC + AUC (desno).



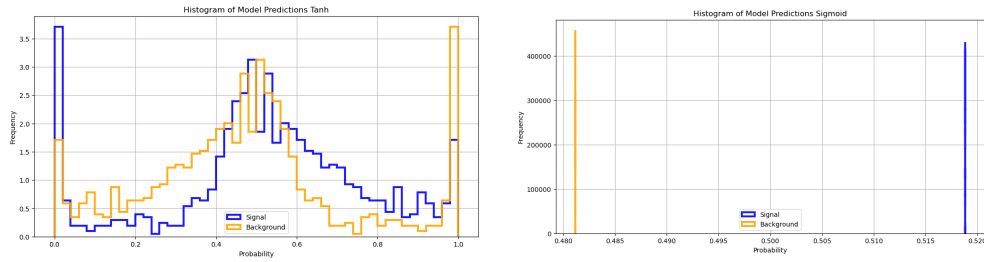
Slika 9: Prikaz histograma ocene.

5.3 Primerjava optimizatorjev

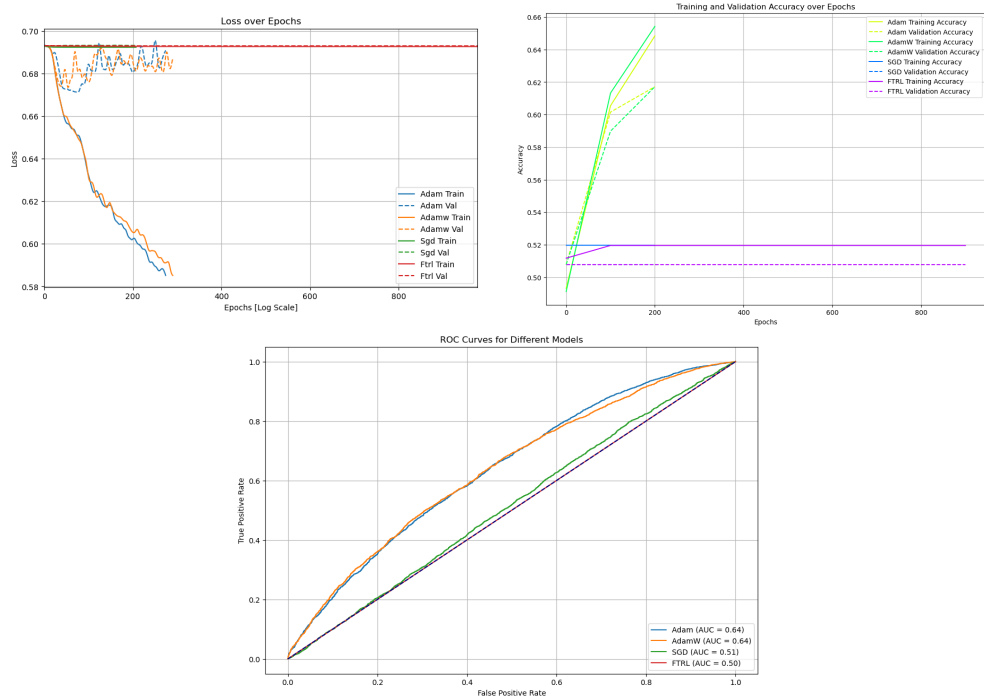
Pri optimizatorjih sem primerjal nekaj najbolj znanih: SGD, Adam, AdamW in FTLR, prikazano na sliki 11. SGD, čeprav preprost in učinkovit, pogosto zahteva natančno nastavljanje hitrosti učenja in se je izkazal kot najslabši v vseh testih. Adam in njegove različice (AdamW, AdamMax) običajno nudijo hitrejšo konvergenco in boljšo uspešnost z prilagajanjem hitrosti učenja in vključevanjem momentuma. FTLR, novejši optimizator, se je izkazal kot neka vmesna odločitev, vendar je to lahko posledica mojega slabšega razumevanja kako pravilno nastaviti ta optimizator.

Zmanjkalo mi je časa, da bi uspešno pogruntal kje se nahaja napaka v kodi.

5.4. Osnovne in izpeljane spremenljivke



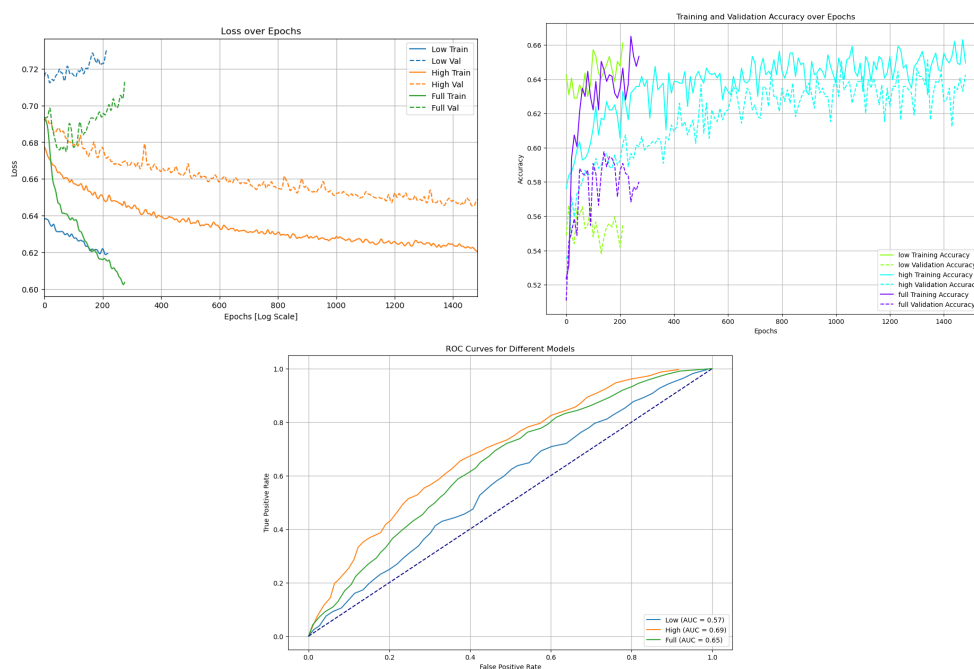
Slika 10: Prikaz histograma ocene.



Slika 11: Prikaz izgube (levo), točnosti (sredina) in ROC + AUC (desno).

5.4 Osnovne in izpeljane spremenljivke

Želel sem ponoviti rezultate iz [2] - primerjavo uspešnosti mrež glede na izbiro vhodnih spremenljivk. Uporabil sem tri nabore spremenljivk: enega 21 osnovnimi spremenljivkami, drugega z samo izpeljanimi spremenljivkami. Tako sem storil saj so v [2] uporabili tudi diskretne in binarne spremenljivke. Za ostale hiperparametre sem uporabil srednje velik model, Elu aktivacijo, ter Adam optimizator. Rezultati so prikazani na grafu 12. Razvidno je da model se večinoma uči iz izpeljanih spremenljivk, saj ne doseže velike razlike v primerjavi s polnimi podatki, samo osnovne spremenljivke pa ne zadoščajo. Tudi vidimo, iz slike 13, da je histogram bližje željeni obliki.



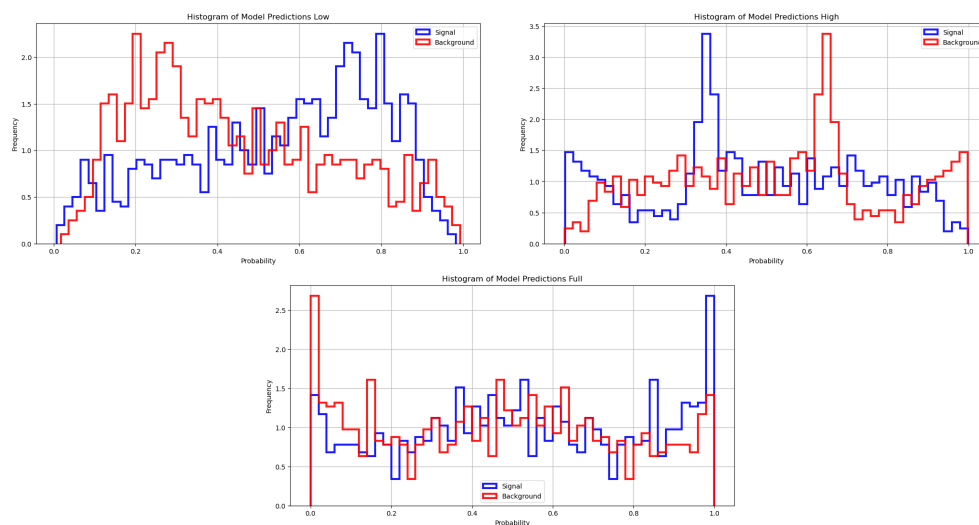
Slika 12: Prikaz izgube (levo), točnosti (sredina) in ROC + AUC (desno).

6 Pospešena odločitvena drevesa (BDT)

Odločitvena drevesa delujejo tako, da ustvarijo zaporedje vprašanj, ki usmerjajo model k napovedi oznake. Vsako vprašanje primerja značilnost z mejno vrednostjo, kar vodi do veje v drevesu. Ta postopek se nadaljuje, dokler ne pridemo do lista, ki poda napovedano oznako. Odločitvena drevesa so lažje razumljiva in vizualizirana v primerjavi z nevronskimi mrežami, ki so pogosto obravnavane kot "črne škatle". Vendar pa se odločitvena drevesa lahko hitro pretrenirajo. Proces učenja vključuje določitev optimalnih mejnih vrednosti na podlagi funkcij nečistosti, kot so cross-entropy, Gini indeks ali napaka pri klasifikaciji. Od svojega začetka v osemdesetih letih prejšnjega stoletja so se odločitvena drevesa razvila v bolj sofisticirane metode, kot so izboljšana odločitvena drevesa (BDT). BDT združujejo več dreves, pri čemer vsako popravi napake prejšnjega, z priljubljenimi algoritmi, kot so AdaBoost, Gradient Boosting in CatBoost, ki sem ga jaz uporabil.

6.1 Primerjava velikosti

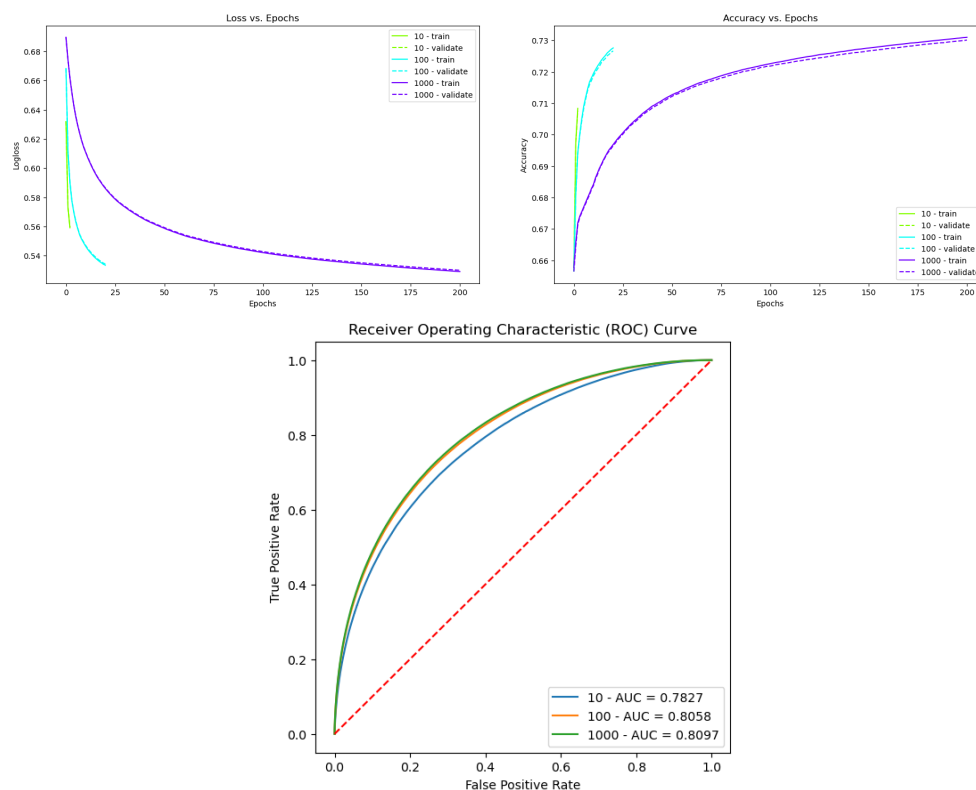
Enako kot pri nevronskih mrežah sem tudi tukaj spreminjal hiperparametre z namenom iskanja optimalne konfiguracije. Iz grafov 14 je očitno da z večanjem števila dreves in globino le teh dobimo boljše rezultate, vendar za ceno vse manjših donosov.



Slika 13: Prikaz histograma ocene za osnovne (levo), izpeljane (sredina) in polne spremenljivke (desno).

6.2 Osnovne in izpeljane spremenljivke

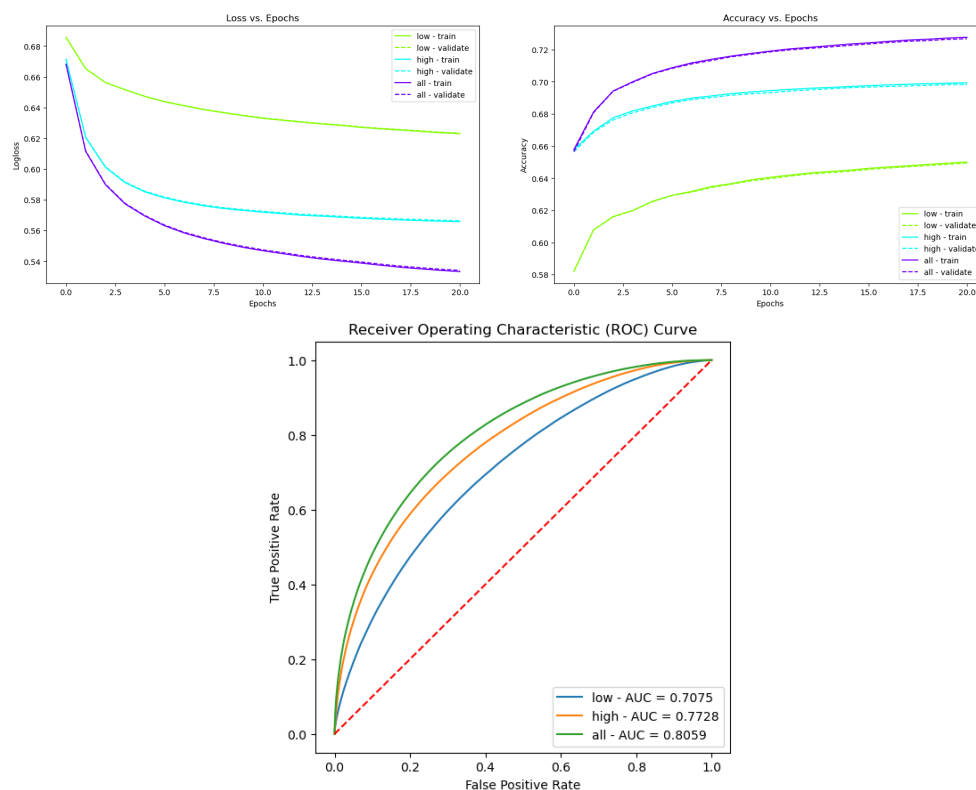
Tudi pri odločitvenih drevesih sem preveril uporabo določenih spremenljivk za treniranje ter dobil rezultate, ki se ujemajo / potrjujejo hipotezo iz prejšnjega poglavja [15](#), [16](#).



Slika 14: Prikaz izgube (levo), točnosti (sredina) in ROC + AUC (desno).

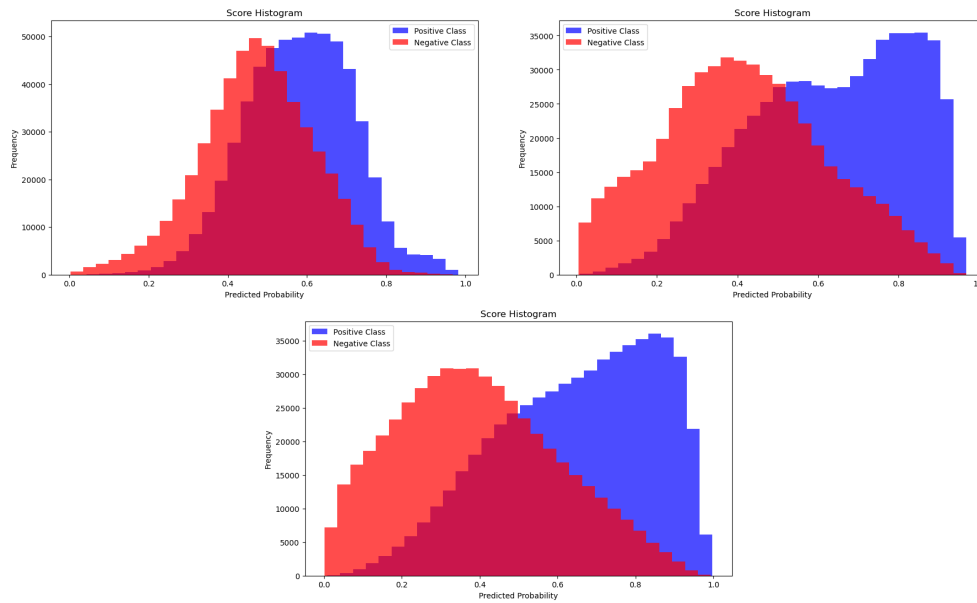
6.3 Optimalno drevo

Zdaj ko imamo ostale rezultate, lahko implementiramo še optimalen model, za čim višjo kvaliteto kvalifikacije [17](#).



Slika 15: Prikaz izgube (levo), točnosti (sredina) in ROC + AUC (desno).

Odločitvena drevesa lahko tudi prikažemo in s tem pridobimo globlje razumevanje v proces odločanja. Na sliki 18 je priakzan en primer takega drevesa, ki je pa plitek, saj je večje modele težko predstaviti na majhnem platnu.



Slika 16: Prikaz ocene osnovne spremenljivke (levo), izpeljane spremenljivke (sredina) in polno (desno).

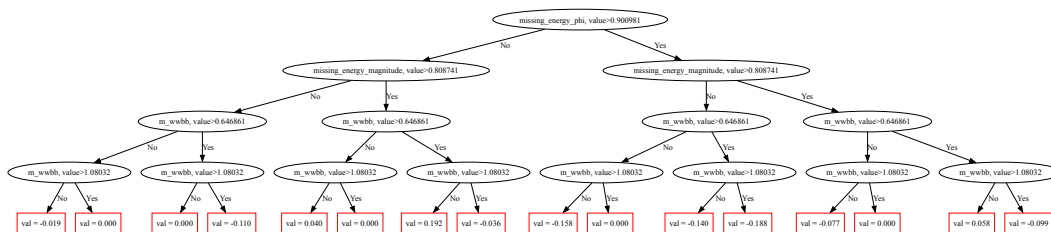
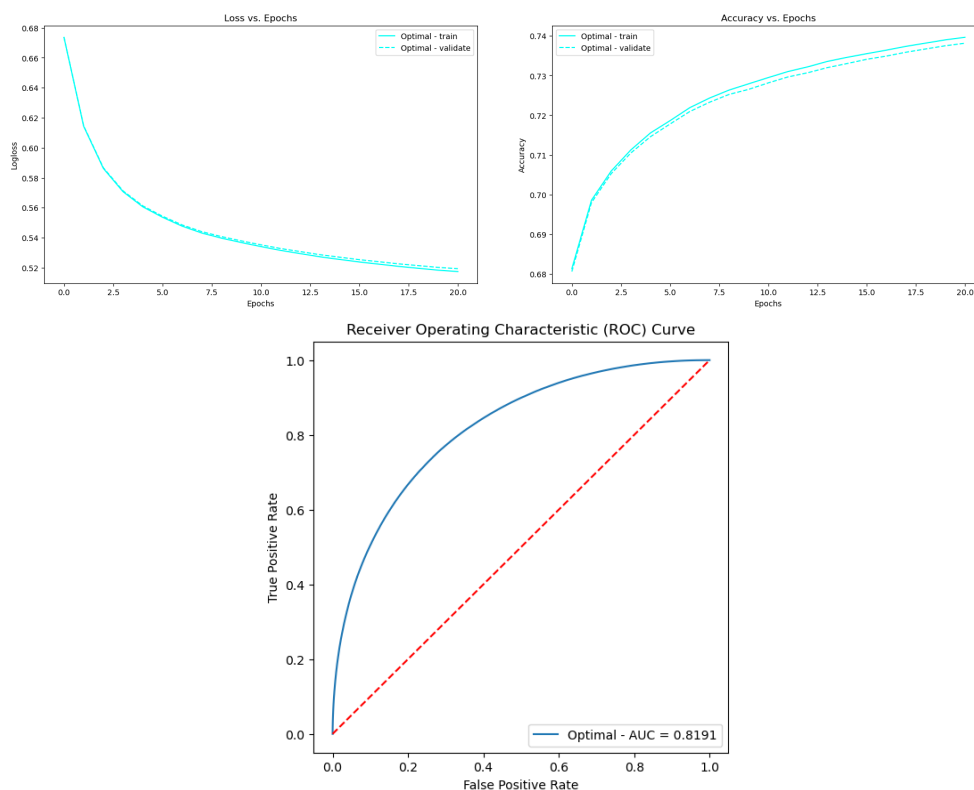
7 PyTorch implementacija

Ker sem bil izjemno razočaran nad rezultati in mojo implementacijo v TensorFlow okvirju, nisem mogel v dobri veri predstaviti nedokončanega projekta. Zato sem hitro (s pomočjo ChatGPT) preuredil kodo, da deluje v okviru PyTorch. Takšna hitra prilagoditev je bila mogoča le zaradi uporabe GPU znotraj okvira PyTorch.

Zopet sem izvedel tri sklope eksperimentov:

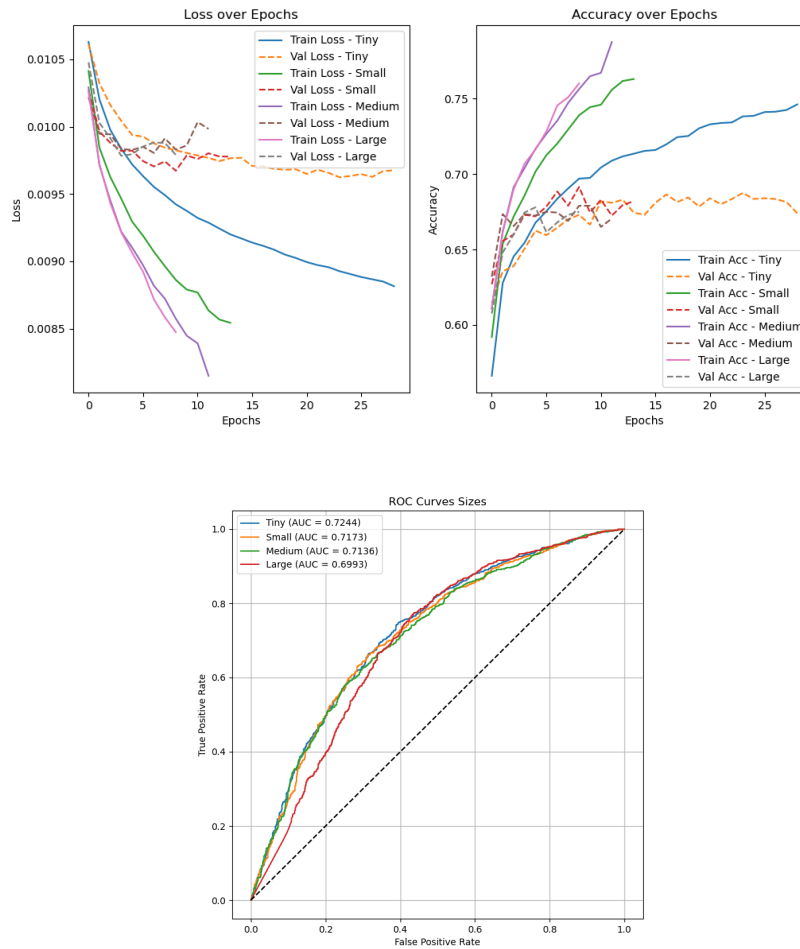
- **Spreminjanje velikosti modela:** Treniral sem modele z različnimi globinami: majhen, srednji in velik. Vsak model je imel izhodno plast s funkcijo softmax za klasifikacijo signalnih in ozadnih dogodkov. Za preprečevanje prenaučitve smo uporabili zgodnje ustavljanje. Rezultati so prikazani na sliki [19](#).
- **Aktivacijske funkcije:** Ocenil sem vpliv različnih aktivacijskih funkcij (ReLU, ELU, Tanh in Sigmoid) pri modelih z enako arhitekturo (srednji model) [20](#).

Tukaj je še bolj razvidno, da sta Elu in ReLu funkciji malo boljši od TanH in Sigmoid, vendar ne drastično.



- **Optimizatorji:** Primerjal sem različne optimizatorje, vključno z SGD, Adam, AdamW, AdamMax in FTLR, da bi ocenil zmogljivost in konvergenco 21.

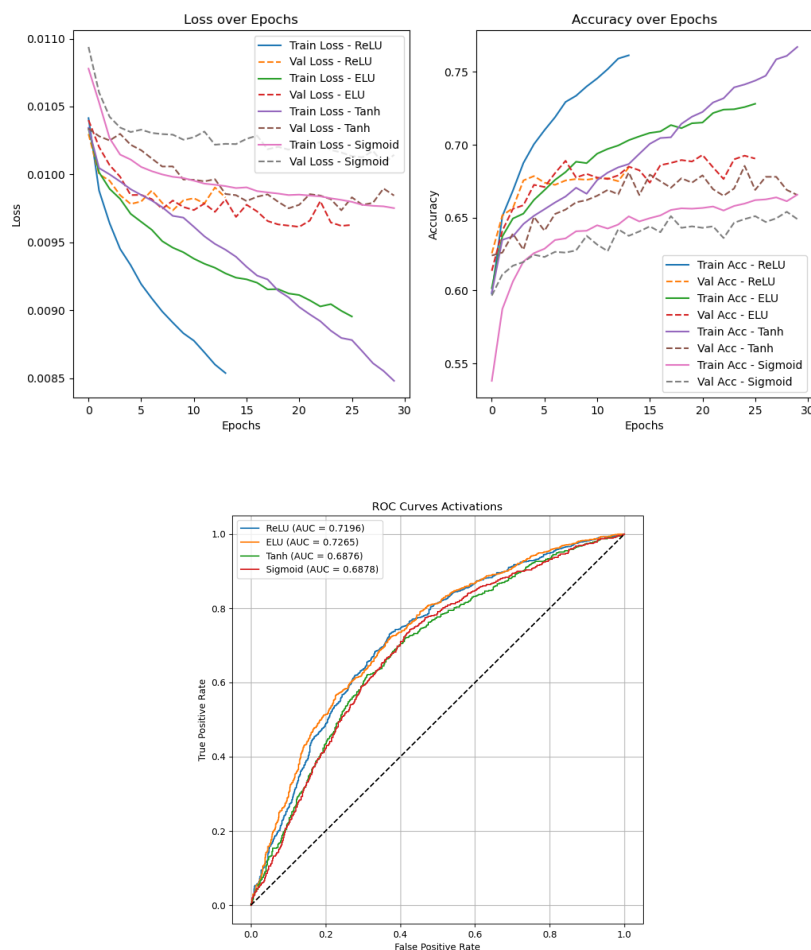
Tukaj je iz grafov razvidno, da je velika razlika med uporabo preprostega algoritma kot je Stohastični Gradientni Spust (SGD) veliko slabše kot uporaba bolj kvalitetnega algoritma, npr.: Adam.



Slika 19: Prikaz izgube (levo), točnosti (sredina) in ROC + AUC (desno).

7.1 Optimalen

Za konec sem zopet implementiral najbolj optimalen model, tokrat tudi treniral na večji količini podatkov (100.000). Rezultati so prikazani na slikah ?? ter ??. Skonstruiran je kot kombinacija srednje velikega modela, aktivacijske funkcije Elu ter optimizatorja Adam.

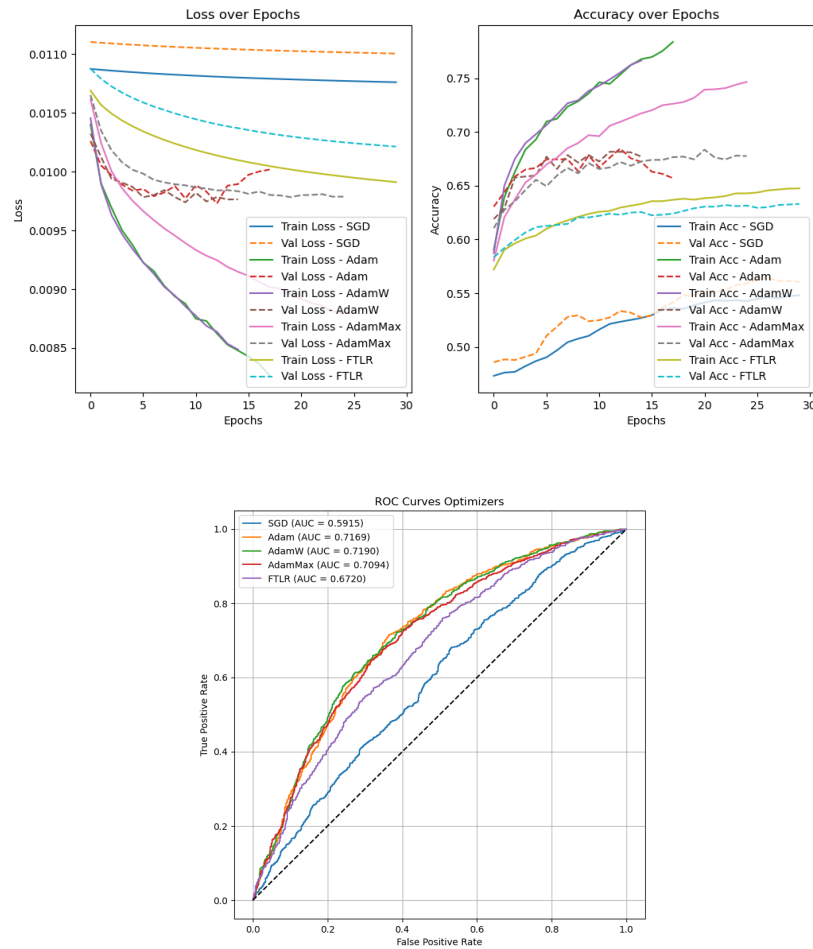


Slika 20: Prikaz izgube (levo), točnosti (sredina) in ROC + AUC (desno).

8 Zaključek

V tej nalogi sem raziskoval tako nevronske mreže kot odločitvena drevesa. Poglobil sem se v različne priljubljene okvire za nevronske mreže in odkril pogoste pasti, povezane z vsakim od njih. To je omogočilo globlje razumevanje osnov—medtem ko je razumevanje enačb na papirju ena stvar, je njihova implementacija v kodi in optimizacija za doseganje najboljše uspešnosti povsem drugačna izkušnja. Presenetljivo je, da kljub relativni preprostosti odločitvenih dreves, dosežejo uspešnost primerljivo z nevronskimi mrežami, ali celo boljše, če ne zgradimo nevronske mreže dovolj previdno, pri tem pa so bila veliko bolj zabavna za delo, čeprav sem sprva pozabil na njihovo prisotnost v nalogi.

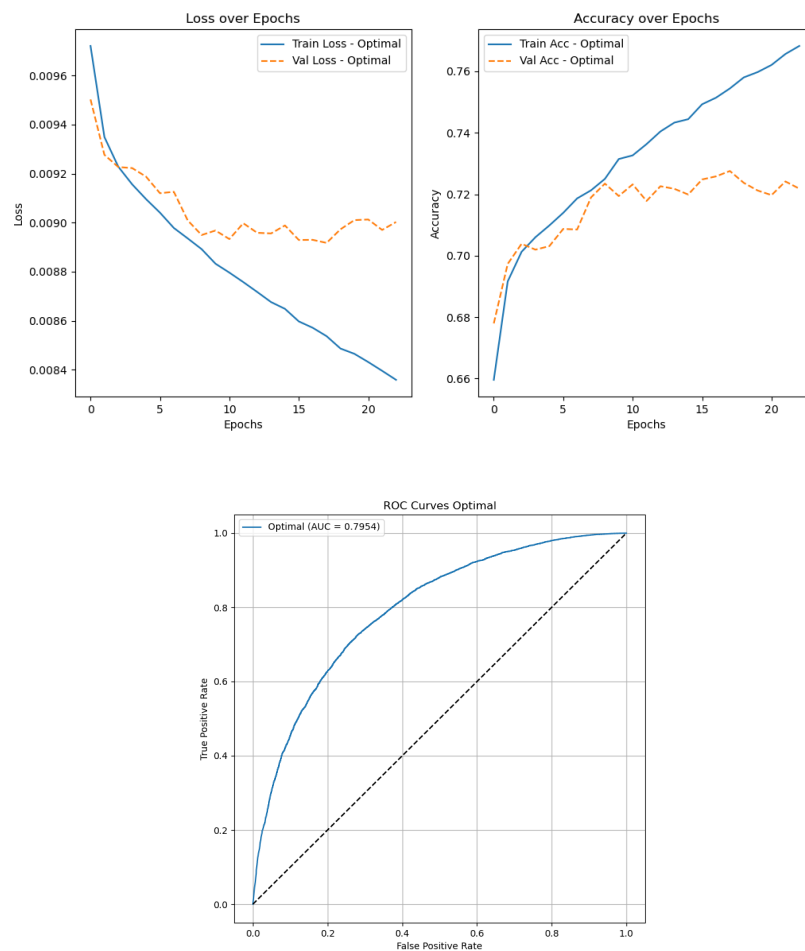
Projekt sem začel v drugem tednu julija, vendar je zaradi pretirano visokih temperatur moj računalnik segrel CPU do 85°C v samo dveh minutah in se popolnoma sesul (tudi v mirovanju). Posledično sem kupil nov računalnik z močnim GPU-jem, da bi olajšal treniranje in svoje življenje. Kljub temu sem



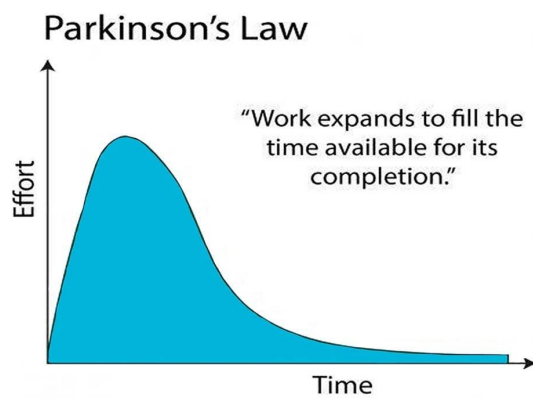
Slika 21: Prikaz izgube (levo), točnosti (sredina) in ROC + AUC (desno).

žal dal prednost projektom, povezanim z iskanjem zaposlitve, kar je privedlo do trenutne situacije, da ponovno pišem to nalogo pozno v noč. Ko se pripravljam, da grem spat za nekaj ur preden se odpravim v CERN, Vas zapuščam z eno miseljo, ki odlično povzema moje izkušnje s tem predmetom:

Parkinsonov zakon ste nam na predavanjih predstavili tudi Vi, prijatelj je pa še lepše ubesedil "Ta predmet ti vzame vsako sekundo časa ki mu jo dopustiš."



Slika 22: Prikaz izgube (levo), točnosti (sredina) in ROC + AUC (desno).



Literatura

- [1] CERN. *D. Whiteson. HIGGS. UCI Machine Learning Repository*. URL: <https://mlphysics.ics.uci.edu/data/higgs/> (visited on 09/05/2024).
- [2] P. Whiteson D P. Baldi P. Sadowski. “Searching for exotic particles in high-energy physics with deep learning.” In: *Nat Commun* (2014). DOI: <https://doi.org/10.1038/ncomms5308>. URL: <https://www.nature.com/articles/ncomms5308>.
- [3] Elijan Mastnak. *Convolutional Neural Networks and Particle Physics*. URL: <https://ejmastnak.com/projects/seminar/> (visited on 09/05/2024).
- [4] Adam Savage Jamie Hyneman NVIDIA. *Mythbusters Demo GPU versus CPU*. URL: <https://www.youtube.com/watch?v=-P28LKWTzrI> (visited on 09/05/2024).